

# **Udacity Project 1**

## **Building an Azure Data Warehouse for Bike Share Data Analytics**

# Task 1: Create your Azure resources

---

- **Create an Azure Database for PostgreSQL:**
  1. Search Azure Database for PostgreSQL
  2. Click on "Create"
  3. Select "Subscription" and "Resource type"
  4. Enter Server Name
  5. Choose Region Same as Resource Group
  6. In Workload Type: Select "Development"
  7. Select Authentication type: PostgreSQL Authentication
  8. Enter "USERNAME" and "PASSWORD"
  9. Click "Next → Networking"
  10. Choose Connectivity Method: Public Access
  11. Firewall Rule: Allow Public Access from Any Azure Service within This Server
  12. Click "Add Current IP Address"
  13. Click "Next → Security"
  14. Data Encryption Key: Service Managed Key
  15. Click "Review and Create"
  16. After Validation Passed Successfully Click "Create".

- **Create a Storage Account:**

1. Search "Storage Account"
2. Click "Create"
3. Select "Subscription" and "Resource Group"
4. Select Region
5. Performance: Standard
6. Redundancy: GRS
  - Mark Read Access data available in the event of region availability: Check On
7. Next → Advance
8. Datalake Storage Gen2:
  - Enable Hierarchical Namespace: Check On
9. Next → Networking
10. Enable public access from all network
11. Next → Encryption
12. Encryption type: MMK
13. Click "Review and Create"
14. When Validation is Passed Click "Create"
15. Now Create a Container after deploying of storage

- **Create Azure Synapse Workplace:**

- Search “Azure Synapse Analytics”
- Click “Create”
- Enter Workspace Name
- Select Region same as Resource Group
- Select DataLake Storage Gen2:  
Account Name: “Enter Storage Account Name”  
File System Name: “Enter Container Name”
- Next → Security
- Authentication Method: Use both local and Azure SQL  
Server admin Login  
→ Enter Username and password
- Next → Networking
- Managed Virtual Network: Disable
- Allow connection from all IP Address: Check On
- Click “Review and Create”
- After validation Passed Successfully Click “Create”

# Task\_1 Figure:

The screenshot shows the Microsoft Azure portal interface. At the top, there is a navigation bar with the Microsoft Azure logo, a search bar, and a user profile dropdown. The user profile dropdown is highlighted with a red box and shows the email address 'ritikkumar7280@gmail.com' and the text 'DEFAULT DIRECTORY (RITIKKUM...)'. Below the navigation bar, the main content area displays the 'Project\_1' resource group overview. The left sidebar contains a list of navigation items: Overview, Activity log, Access control (IAM), Tags, Resource visualizer, Events, Settings, Cost Management, Monitoring, Automation, and Help. The main content area is divided into two sections: 'Essentials' and 'Resources'. The 'Essentials' section shows subscription information, including the subscription name 'Azure subscription 1', subscription ID '67f9d7d3-3388-430b-bc3c-34b76ac70782', and location 'Canada Central'. The 'Resources' section shows a list of resources, including 'projectserver1', 'projectstorageritik1', and 'projectsynapse-1'. A red box highlights the resources table, which has columns for Name, Type, and Location. The table lists three resources, all located in Canada Central.

| Name                 | Type                                          | Location       |
|----------------------|-----------------------------------------------|----------------|
| projectserver1       | Azure Database for PostgreSQL flexible server | Canada Central |
| projectstorageritik1 | Storage account                               | Canada Central |
| projectsynapse-1     | Synapse workspace                             | Canada Central |

....Complete....

# Task 2: Design a star schema

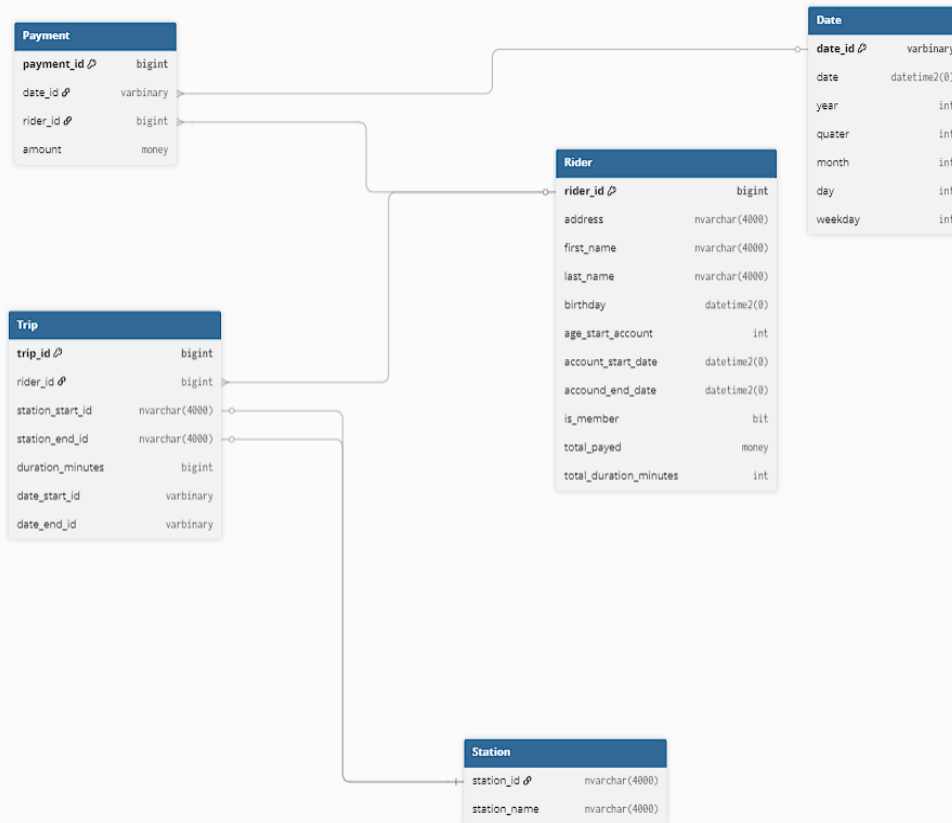
---

SQL code to create schema:

```
Table fact_trip {
  trip_id bigint [primary key]
  rider_id bigint
  station_start_id nvarchar(4000)
  station_end_id nvarchar(4000)
  duration_minutes bigint
  date_start_id varbinary
  date_end_id varbinary
}
Table fact_payment {
  payment_id bigint [primary key]
  date_id varbinary
  rider_id bigint
  amount money
}
Table dim_rider {
  rider_id bigint [primary key]
  address nvarchar(4000)
  first_name nvarchar(4000)
  last_name nvarchar(4000)
  birthday datetime2(0)
  age_start_account int
  account_start_date datetime2(0)
  account_end_date datetime2(0)
  is_member bit
  total_payed money
  total_duration_minutes int
}
Table dim_date {
  date_id varbinary [primary key]
  date datetime2(0)
  year int
  quarter int
  month int
  day int
  weekday int
}
Table dim_station{
  station_id nvarchar(4000)
  station_name nvarchar(4000)
}
Ref: fact_trip.rider_id > dim_rider.rider_id
Ref: fact_payment.rider_id > dim_rider.rider_id
```

Ref: fact\_payment.date\_id > dim\_date.date\_id  
Ref: fact\_trip.station\_start\_id - dim\_station.station\_id  
Ref: fact\_trip.station\_end\_id - dim\_station.station\_id

I use <https://dbdiagram.io/> to create database diagram.



....Complete....

# Task 3: Create the data in PostgreSQL

---

➔ Add the host, username, and password information for my PostgreSQL database in [Github: ProjectDataToPostgres.py](#)

```
import psycopg2
from psycopg2.extensions import ISOLATION_LEVEL_AUTOCOMMIT

#####
# Update connection string information #
#####
host = "projectserver1.postgres.database.azure.com"
user = "ritik"
password = "asdfghjkl321@"

# Create a new DB
sslmode = "require"
dbname = "postgres"
conn_string = "host={0} user={1} dbname={2} password={3} sslmode={4}".format(host, user,
dbname, password, sslmode)
conn = psycopg2.connect(conn_string)
conn.set_isolation_level(ISOLATION_LEVEL_AUTOCOMMIT)
print("Connection established")

cursor = conn.cursor()
cursor.execute('DROP DATABASE IF EXISTS udacityproject')
cursor.execute("CREATE DATABASE udacityproject")
# Clean up initial connection
conn.commit()
cursor.close()
conn.close()

# Reconnect to the new DB
dbname = "udacityproject"
conn_string = "host={0} user={1} dbname={2} password={3} sslmode={4}".format(host, user,
dbname, password, sslmode)
conn = psycopg2.connect(conn_string)
print("Connection established")
cursor = conn.cursor()

# Helper functions
def drop_recreate(c, tablename, create):
    c.execute("DROP TABLE IF EXISTS {0};".format(tablename))
    c.execute(create)
```



```

    print("Finished creating table {0}".format(tablename))

def populate_table(c, filename, tablename):
    f = open(filename, 'r')
    try:
        cursor.copy_from(f, tablename, sep=",", null = "")
        conn.commit()
    except (Exception, psycopg2.DatabaseError) as error:
        print("Error: %s" % error)
        conn.rollback()
        cursor.close()
    print("Finished populating {0}".format(tablename))

# Create Rider table
table = "rider"
filename = 'C:/Users/ritik/Desktop/Project_1/Azure-Data-Warehouse-Project-main/Azure-Data-Warehouse-Project-main/starter/data/riders.csv'
create = "CREATE TABLE rider (rider_id INTEGER PRIMARY KEY, first VARCHAR(50), last VARCHAR(50), address VARCHAR(100), birthday DATE, account_start_date DATE, account_end_date DATE, is_member BOOLEAN);"

drop_recreate(cursor, table, create)
populate_table(cursor, filename, table)

# Create Payment table
table = "payment"
filename = 'C:/Users/ritik/Desktop/Project_1/Azure-Data-Warehouse-Project-main/Azure-Data-Warehouse-Project-main/starter/data/payments.csv'
create = "CREATE TABLE payment (payment_id INTEGER PRIMARY KEY, date DATE, amount MONEY, rider_id INTEGER);"

drop_recreate(cursor, table, create)
populate_table(cursor, filename, table)

# Create Station table
table = "station"
filename = 'C:/Users/ritik/Desktop/Project_1/Azure-Data-Warehouse-Project-main/Azure-Data-Warehouse-Project-main/starter/data/stations.csv'
create = "CREATE TABLE station (station_id VARCHAR(50) PRIMARY KEY, name VARCHAR(75), latitude FLOAT, longitude FLOAT);"

drop_recreate(cursor, table, create)
populate_table(cursor, filename, table)

# Create Trip table
table = "trip"
filename = 'C:/Users/ritik/Desktop/Project_1/Azure-Data-Warehouse-Project-main/Azure-Data-Warehouse-Project-main/starter/data/trips.csv'

```

```
create = "CREATE TABLE trip (trip_id VARCHAR(50) PRIMARY KEY, rideable_type VARCHAR(75),  
start_at TIMESTAMP, ended_at TIMESTAMP, start_station_id VARCHAR(50), end_station_id  
VARCHAR(50), rider_id INTEGER);"
```

```
drop_recreate(cursor, table, create)  
populate_table(cursor, filename, table)
```

```
# Clean up  
conn.commit()  
cursor.close()  
conn.close()  
  
print("All done!")
```

The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the terminal at the bottom. The file explorer shows a project structure with folders like 'AZURE-DATA-WAREHOUSE...' and 'starter', and files like 'payments.csv', 'riders.csv', 'stations.csv', and 'trips.csv'. The main editor area shows a Python script named 'ProjectDataToPostgres.py' with the following code:

```
6 #####  
7 host = "projectserver1.postgres.database.azure.com"  
8 user = "ritik"  
9 password = "asdfghjk1321@"  
10  
11 # Create a new DB  
12 sslmode = "require"  
13 dbname = "postgres"  
14 conn_string = "host={0} user={1} dbname={2} password={3} sslmode={4}".format(host, user, dbname, password, sslmode)  
15 conn = psycopg2.connect(conn_string)
```

The terminal at the bottom shows the output of running the script:

```
PS C:\Users\ritik\Desktop\Project_1\Azure-Data-Warehouse-Project-main> python -u "c:\Users\ritik\Desktop\Project_1\Azure-Data-Warehouse-Project-main\Azure-Data-Warehouse-Project-main\starter\ProjectDataToPostgres.py"  
Connection established  
Connection established  
Finished creating table rider  
Finished populating rider  
Finished creating table payment  
Finished populating payment  
Finished creating table station  
Finished populating station  
Finished creating table trip  
Finished populating trip  
All done!  
PS C:\Users\ritik\Desktop\Project_1\Azure-Data-Warehouse-Project-main>
```

....Complete....

# Task 4: EXTRACT the data from PostgreSQL

We will use the ingest wizard to create a one-time pipeline that ingests the data from PostgreSQL into Azure Blob Storage. This will result in all four tables being represented as text files in Blob Storage, ready for loading into the data warehouse.

Here we copy out selected data to storage account from PostgreSQL:

The screenshot displays the 'Copy Data tool' in the Microsoft Azure Synapse Analytics environment. The interface is divided into a sidebar on the left and a main workspace. The sidebar contains a vertical list of steps: Properties (checked), Source (active), Dataset, Configuration, Destination, Settings, and Review and finish. The main workspace is titled 'Copy Data tool' and features a search bar 'Filter by name...' and a list of tables: 'public.payment', 'public.rider', 'public.station', and 'public.trip'. The 'public.payment' table is selected and highlighted with a red box. To the right of the table list is a 'Preview data' button. At the bottom of the workspace, there are navigation buttons: '< Previous', 'Next >', and 'Cancel'. The top bar of the interface shows the Microsoft Azure logo, the path 'Synapse Analytics > projectsynapse-1', and the user profile 'ritikkumar7280@gmail.com' with the role 'DEFAULT DIRECTORY'.

Our Data deployment from PostgreSQL database to Blob storage is succeeded:

Microsoft Azure | Synapse Analytics | projectsynapse-1

Copy Data tool

Properties

Source

Destination

Settings

Review and finish

Review

Deployment

Azure Database for PostgreSQL → Azure Blob Storage

Deployment complete

| Deployment step                     | Status      |
|-------------------------------------|-------------|
| Validating copy runtime environment | ✓ Succeeded |
| > Creating datasets                 | ✓ Succeeded |
| > Creating pipelines                | ✓ Succeeded |
| > Running pipelines                 | ✓ Succeeded |

Datasets and pipelines have been created. You can now monitor and edit the copy pipelines or click finish to close Copy Data Tool.

Finish Edit pipeline Monitor

Now our deployed database is shows in Storage account in text(.txt) Format:

Microsoft Azure | Upgrade

Search resources, services, and docs (G+/)

Copilot

ritikkumar7280@gmail.com

Home > Project\_1 > projectstorageritik1 | Containers >

projectcontainer1

Search

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Shared access tokens

Manage ACL

Access policy

Properties

Metadata

projectcontainer1

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 4 items

| <input type="checkbox"/> | Name               | Last modified        | Access tier    | Blob type  | Size       | Lease state |     |
|--------------------------|--------------------|----------------------|----------------|------------|------------|-------------|-----|
| <input type="checkbox"/> | public.payment.txt | 27/08/2025, 19:32:58 | Hot (Inferred) | Block blob | 88.08 MiB  | Available   | ... |
| <input type="checkbox"/> | public.rider.txt   | 27/08/2025, 19:32:52 | Hot (Inferred) | Block blob | 8.44 MiB   | Available   | ... |
| <input type="checkbox"/> | public.station.txt | 27/08/2025, 19:32:51 | Hot (Inferred) | Block blob | 61.38 KiB  | Available   | ... |
| <input type="checkbox"/> | public.trip.txt    | 27/08/2025, 19:33:27 | Hot (Inferred) | Block blob | 524.68 MiB | Available   | ... |

....Completed....

# Task 5: LOAD the data into external tables in the data warehouse

---

- Navigate to “Data” in the Azure Synapse Analytics.
  - Here we can see our synapse workplace that contain blob storage, Data Lake Gen2 storage, and uploaded files.
  - Create an SQL Database:
    - Click on “+” and select “SQL database”.
  - Create external tables:
    - Click on Workplace
    - Click on newly created SQL database → External table → Three Dots(...) → New SQL Script
  - Here in this empty SQL script we write out SQL code to create external table.
  - Below I paste the code namely.....
- ❖ CREATE EXTERNAL TABLE [dbo.staging\_payment]

```
IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name =  
'SynapseDelimitedTextFormatWithHeader')
```

```
CREATE EXTERNAL FILE FORMAT [SynapseDelimitedTextFormatWithHeader]  
WITH (  
    FORMAT_TYPE = DELIMITEDTEXT,  
    FORMAT_OPTIONS (  
        FIELD_TERMINATOR = ',',  
        -- Skip header row  
        FIRST_ROW = 2,  
        USE_TYPE_DEFAULT = FALSE  
    )  
);  
GO
```

```
-- External data source (only create if it doesn't already exist)  
IF NOT EXISTS (SELECT * FROM sys.external_data_sources WHERE name =  
'projectcontainer1_projectstorageritik1_dfs_core_windows_net')  
    CREATE EXTERNAL DATA SOURCE  
[projectcontainer1_projectstorageritik1_dfs_core_windows_net]
```

```

        WITH (
            LOCATION = 'abfss://projectcontainer1@projectstorageritik1.dfs.core.windows.net'
        );
GO

-- Drop existing external table if it exists
IF OBJECT_ID('dbo.staging_payment', 'U') IS NOT NULL
    DROP EXTERNAL TABLE dbo.staging_payment;
GO

-- Create external table mapped to the file (header skipped)
CREATE EXTERNAL TABLE [dbo].[staging_payment] (
    [payment_id] BIGINT,
    [date] DATETIME2,
    [amount] MONEY,
    [rider_id] BIGINT
)
WITH (
    LOCATION = '/public.payment.txt',
    DATA_SOURCE = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT = [SynapseDelimitedTextFormatWithHeader]
);
GO

-- Test query
SELECT TOP 10 * FROM dbo.staging_payment;
GO

```

## ❖ CREATE EXTERNAL TABLE [dbo.staging\_rider]

```

-- Drop old external table if exists
IF OBJECT_ID('dbo.staging_rider', 'U') IS NOT NULL
    DROP EXTERNAL TABLE dbo.staging_rider;
GO

-- External table with all NVARCHAR columns (raw load)
CREATE EXTERNAL TABLE [dbo].[staging_rider] (
    [rider_id] NVARCHAR(100),
    [first] NVARCHAR(4000),
    [last] NVARCHAR(4000),
    [address] NVARCHAR(4000),
    [birthday] NVARCHAR(100),
    [account_start_date] NVARCHAR(100),
    [account_end_date] NVARCHAR(100),
    [is_member] NVARCHAR(10)
)

```

```

)
WITH (
    LOCATION = 'public.rider.txt',
    DATA_SOURCE = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT = [SynapseDelimitedTextFormat]
);
GO

-- Now query with type casting
SELECT TOP 10
    TRY_CAST(rider_id AS BIGINT) AS rider_id,
    first,
    last,
    address,
    TRY_CAST(birthday AS DATETIME2(0)) AS birthday,
    TRY_CAST(account_start_date AS DATETIME2(0)) AS account_start_date,
    TRY_CAST(account_end_date AS DATETIME2(0)) AS account_end_date,
    TRY_CAST(is_member AS BIT) AS is_member
FROM dbo.staging_rider
WHERE rider_id <> 'rider_id';  -- skips header row if present
GO

```

## ❖ CREATE EXTERNAL TABLE [dbo.staging\_station]

```

IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name =
'SynapseDelimitedTextFormat')
    CREATE EXTERNAL FILE FORMAT [SynapseDelimitedTextFormat]
    WITH ( FORMAT_TYPE = DELIMITEDTEXT ,
        FORMAT_OPTIONS (
            FIELD_TERMINATOR = ',',

            -- skip header row
            FIRST_ROW = 1,
            USE_TYPE_DEFAULT = FALSE
        ))
GO

IF NOT EXISTS (SELECT * FROM sys.external_data_sources WHERE name =
'projectcontainer1_projectstorageritik1_dfs_core_windows_net')
    CREATE EXTERNAL DATA SOURCE
[projectcontainer1_projectstorageritik1_dfs_core_windows_net]
    WITH (
        LOCATION = 'abfss://projectcontainer1@projectstorageritik1.dfs.core.windows.net'
    )
GO

```

```

-- Drop and recreate external table with same name
IF OBJECT_ID('dbo.staging_station', 'U') IS NOT NULL
    DROP EXTERNAL TABLE dbo.staging_station;
GO

CREATE EXTERNAL TABLE [dbo].[staging_station] (
    [station_id] NVARCHAR(100),
    [name] NVARCHAR(4000),
    [latitude] NVARCHAR(100),
    [longitude] NVARCHAR(100)
)
WITH (
    LOCATION = '/public.station.txt',
    DATA_SOURCE = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT = [SynapseDelimitedTextFormat]
);
GO

-- change in query with casting for safty
SELECT TOP 10
    TRY_CAST(station_id AS BIGINT) AS station_id,
    name,
    TRY_CAST(latitude AS FLOAT) AS latitude,
    TRY_CAST(longitude AS FLOAT) AS longitude
FROM dbo.staging_station;
GO

```

## ❖ CREATE EXTERNAL TABLE [dbo.staging\_station]

```

IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name =
'SynapseDelimitedTextFormat')
    CREATE EXTERNAL FILE FORMAT [SynapseDelimitedTextFormat]
    WITH ( FORMAT_TYPE = DELIMITEDTEXT ,
        FORMAT_OPTIONS (
            FIELD_TERMINATOR = ',',
            USE_TYPE_DEFAULT = FALSE
        ))
GO

IF NOT EXISTS (SELECT * FROM sys.external_data_sources WHERE name =
'projectcontainer1_projectstorageritik1_dfs_core_windows_net')
    CREATE EXTERNAL DATA SOURCE
[projectcontainer1_projectstorageritik1_dfs_core_windows_net]
    WITH (
        LOCATION = 'abfss://projectcontainer1@projectstorageritik1.dfs.core.windows.net'
    )
GO

```

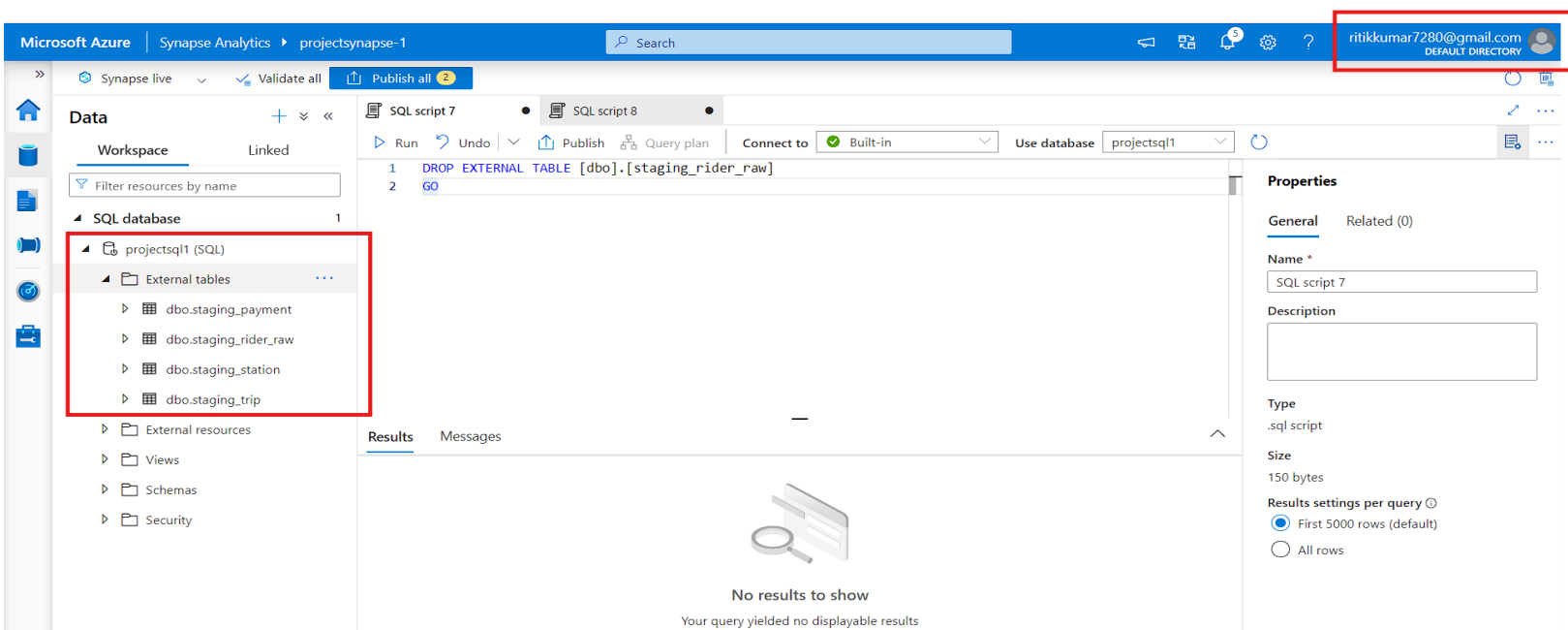


```

CREATE EXTERNAL TABLE [dbo].[staging_trip] (
    [trip_id] nvarchar(4000),
    [rideable_type] nvarchar(4000),
    [start_at] varchar(50),
    [ended_at] varchar(50),
    [start_station_id] nvarchar(4000),
    [end_station_id] nvarchar(4000),
    [rider_id] varchar(50)
)
WITH (
    LOCATION = '/public.trip.txt',
    DATA_SOURCE = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT = [SynapseDelimitedTextFormat]
)
GO
SELECT TOP 10 * FROM [dbo].[staging_trip]
GO

```

These tables are going to be created in our database when we run the above SQL scripts: -



When we run the “DROP” command, that table is removed from the database:

```
DROP EXTERNAL TABLE {dbo}.{staging_rider_raw} GO
```

Microsoft Azure | Synapse Analytics | projectsynapse-1

SQL script 7

```
1 DROP EXTERNAL TABLE [dbo].[staging_rider_raw]
2 GO
```

Properties

General

Name \*  
SQL script 7

Description

Type  
.sql script

Size  
150 bytes

Results settings per query ⓘ  
☒ First 5000 rows (default)  
☐ All rows

Results

No results to show  
Your query yielded no displayable results

Our whole table is uploaded, now we can see those table by:

```
select count(1) as count, 'staging_payment' as Table_Name from staging_payment
union
select count(1) as count, 'staging_rider' as Table_Name from staging_rider
union
select count(1) as count, 'staging_station' as Table_Name from staging_station
union
select count(1) as count, 'staging_trip' as Table_Name from staging_trip
```

Microsoft Azure | Synapse Analytics | projectsynapse-1

SQL script 1

```
1 select count(1) as count, 'staging_payment' as Table_Name from staging_payment
2 union
3 select count(1) as count, 'staging_rider' as Table_Name from staging_rider
4 union
5 select count(1) as count, 'staging_station' as Table_Name from staging_station
6 union
7 select count(1) as count, 'staging_trip' as Table_Name from staging_trip
```

Results

| count   | Table_Name      |
|---------|-----------------|
| 75001   | staging_rider   |
| 839     | staging_station |
| 4584922 | staging_trip    |
| 1946607 | staging_payment |

Properties

General

Name \*  
SQL script 1

Description

Type  
.sql script

Size  
155 bytes

Results settings per query ⓘ  
☒ First 5000 rows (default)  
☐ All rows

....Complete....

# Task 6: TRANSFORM the data to the star schema using CETAS

---

- We will Create a new SQL script, and ensure you are connected to the serverless SQL pool and your SQL database.
- I uploaded the following code to see convert my uploaded data into star schema :

## ❖ CREATE EXTERNAL TABLE dbo.fact\_payment

-- Use the same file format as used for creating the External Tables during the LOAD step.

```
IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name =  
'SynapseDelimitedTextFormat')
```

```
    CREATE EXTERNAL FILE FORMAT [SynapseDelimitedTextFormat]
```

```
    WITH ( FORMAT_TYPE = DELIMITEDTEXT ,
```

```
          FORMAT_OPTIONS (
```

```
            FIELD_TERMINATOR = ',',
```

```
            USE_TYPE_DEFAULT = FALSE
```

```
          ))
```

```
GO
```

-- Use the same data source as used for creating the External Tables during the LOAD step.

-- Storage path where the result set will persist

```
IF NOT EXISTS (SELECT * FROM sys.external_data_sources WHERE name =
```

```
'projectcontainer1_projectstorageritik1_dfs_core_windows_net')
```

```
    CREATE EXTERNAL DATA SOURCE
```

```
[projectcontainer1_projectstorageritik1_dfs_core_windows_net]
```

```
    WITH (
```

```
        LOCATION = 'abfss://projectcontainer1@projectstorageritik1.dfs.core.windows.net'
```

```
    )
```

```
GO
```

```
CREATE EXTERNAL TABLE dbo.fact_payment
```

```
WITH (
```

```
    LOCATION = 'schema_payment',
```

```

        DATA_SOURCE = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
        FILE_FORMAT = [SynapseDelimitedTextFormat]
    )
AS
SELECT [payment_id], [rider_id], hashbytes('md5',CONVERT(VARCHAR(100),[date],126)) as
date_id, [amount]
FROM [dbo].[staging_payment]
GO

SELECT TOP 10 * FROM dbo.fact_payment
GO

```

## ❖ CREATE EXTERNAL TABLE dbo.fact\_trip

```

-- Drop fact_trip if it already exists
IF OBJECT_ID('dbo.fact_trip', 'U') IS NOT NULL
    DROP EXTERNAL TABLE dbo.fact_trip;
GO

-- Create fact_trip with safe casting
CREATE EXTERNAL TABLE dbo.fact_trip
WITH (
    LOCATION      = 'Schema_Trip',
    DATA_SOURCE  = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT   = [SynapseDelimitedTextFormat]
)
AS
SELECT
    TRY_CAST(trip_id AS BIGINT) AS trip_id,
    TRY_CAST(rider_id AS BIGINT) AS rider_id,
    TRY_CAST(start_station_id AS BIGINT) AS start_station_id,
    TRY_CAST(end_station_id AS BIGINT) AS end_station_id,

    -- duration in minutes (only if both dates are valid)
    DATEDIFF(
        MINUTE,
        TRY_CAST(start_at AS DATETIME2),
        TRY_CAST(ended_at AS DATETIME2)
    ) AS duration_minutes,

    -- hashed surrogate keys for start & end dates
    HASHBYTES(
        'MD5',
        CONVERT(VARCHAR(100), TRY_CAST(start_at AS DATETIME2), 126)
    ) AS date_start_id,

```

```

HASHBYTES(
    'MD5',
    CONVERT(VARCHAR(100), TRY_CAST(ended_at AS DATETIME2), 126)
) AS date_end_id

FROM dbo.staging_trip;
GO

-- Check sample
SELECT TOP 10 * FROM dbo.fact_trip;
GO

```

## ❖ CREATE EXTERNAL TABLE dbo.dim\_rider

```

-- Drop dim_rider if exists
IF OBJECT_ID('dbo.dim_rider', 'U') IS NOT NULL
    DROP EXTERNAL TABLE dbo.dim_rider;
GO

-- Create cleaned dim_rider
CREATE EXTERNAL TABLE dbo.dim_rider
WITH (
    LOCATION      = 'schema_rider',
    DATA_SOURCE  = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT   = [SynapseDelimitedTextFormat]
)
AS
SELECT
    TRY_CAST(r.rider_id AS BIGINT) AS rider_id,
    r.[address],
    r.[first],
    r.[last],
    TRY_CAST(r.birthday AS DATE) AS birthday,
    DATEDIFF(
        YEAR,
        TRY_CAST(r.birthday AS DATE),
        TRY_CAST(r.account_start_date AS DATE)
    ) AS age_start_account,
    TRY_CAST(r.account_start_date AS DATE) AS account_start_date,
    TRY_CAST(r.account_end_date AS DATE) AS account_end_date,
    TRY_CAST(r.is_member AS BIT) AS is_member,
    SUM(TRY_CAST(p.amount AS FLOAT)) AS total_payed,
    SUM(
        DATEDIFF(
            MINUTE,

```

```

        TRY_CAST(t.start_at AS DATETIME2),
        TRY_CAST(t.ended_at AS DATETIME2)
    )
) AS total_duration
FROM dbo.staging_rider r
JOIN dbo.staging_payment p
    ON TRY_CAST(r.rider_id AS BIGINT) = TRY_CAST(p.rider_id AS BIGINT)
JOIN dbo.staging_trip t
    ON TRY_CAST(r.rider_id AS BIGINT) = TRY_CAST(t.rider_id AS BIGINT)
GROUP BY
    TRY_CAST(r.rider_id AS BIGINT),
    r.[address],
    r.[first],
    r.[last],
    r.birthday,
    r.account_start_date,
    r.account_end_date,
    r.is_member;
GO

-- Query sample
SELECT TOP 10 * FROM dbo.dim_rider;
GO

```

## ❖ CREATE EXTERNAL TABLE dbo.dim\_station

```

-- Use the same file format as used for creating the External Tables during the LOAD
step.
IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name =
'SynapseDelimitedTextFormat')
    CREATE EXTERNAL FILE FORMAT [SynapseDelimitedTextFormat]
    WITH ( FORMAT_TYPE = DELIMITEDTEXT ,
        FORMAT_OPTIONS (
            FIELD_TERMINATOR = ',',
            USE_TYPE_DEFAULT = FALSE
        ))
GO

-- Use the same data source as used for creating the External Tables during the LOAD
step.
-- Storage path where the result set will persist
IF NOT EXISTS (SELECT * FROM sys.external_data_sources WHERE name =
'projectcontainer1_projectstorageritik1_dfs_core_windows_net')
    CREATE EXTERNAL DATA SOURCE
[projectcontainer1_projectstorageritik1_dfs_core_windows_net]
    WITH (
        LOCATION = 'abfss://projectcontainer1@projectstorageritik1.dfs.core.windows.net'
    )

```

GO

```
CREATE EXTERNAL TABLE dbo.dim_station
WITH (
    LOCATION      = 'Schema_station',
    DATA_SOURCE  = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT   = [SynapseDelimitedTextFormat]
)
AS
SELECT [station_id], [name], latitude, longitude
FROM [dbo].[staging_station]
GO
```

```
SELECT TOP 100 * FROM dbo.dim_station
GO
```

## ❖ CREATE EXTERNAL TABLE dbo.dim\_date

-- Use the same file format as used for creating the External Tables during the LOAD step.

```
IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name =
'SynapseDelimitedTextFormat')
```

```
    CREATE EXTERNAL FILE FORMAT [SynapseDelimitedTextFormat]
    WITH ( FORMAT_TYPE = DELIMITEDTEXT ,
            FORMAT_OPTIONS (
                FIELD_TERMINATOR = ',',
                USE_TYPE_DEFAULT = FALSE
            ))
GO
```

-- Use the same data source as used for creating the External Tables during the LOAD step.

-- Storage path where the result set will persist

```
IF NOT EXISTS (SELECT * FROM sys.external_data_sources WHERE name =
'projectcontainer1_projectstorageritik1_dfs_core_windows_net')
```

```
    CREATE EXTERNAL DATA SOURCE
[projectcontainer1_projectstorageritik1_dfs_core_windows_net]
    WITH (
        LOCATION = 'abfss://projectcontainer1@projectstorageritik1.dfs.core.windows.net'
    )
GO
```

```
CREATE EXTERNAL TABLE dbo.dim_date
WITH (
    LOCATION      = 'Schema_date',
    DATA_SOURCE  = [projectcontainer1_projectstorageritik1_dfs_core_windows_net],
    FILE_FORMAT   = [SynapseDelimitedTextFormat]
```

```

)
AS
SELECT
hashbytes('md5', CONVERT(VARCHAR(100), [date], 126)) as [date_id],
[date],
YEAR([date]) as [year],
DATEPART(QUARTER, [date]) AS [quarter],
MONTH([date]) as [month],
DAY([date]) as [day],
DATEPART(WEEKDAY, [date]) as [weekday]
FROM [dbo].[staging_payment]
GO

```

```

SELECT TOP 100 * FROM dbo.dim_date
GO

```

By running above scripts, we can update our database into star schema.

The screenshot displays the Microsoft Azure Synapse Analytics interface. The top navigation bar shows 'Microsoft Azure | Synapse Analytics | projectsynapse-1'. The left sidebar contains a 'Data' section with 'Workspace' and 'Linked' tabs. Under 'Workspace', there is a 'Filter resources by name' input and a list of 'SQL database' resources. The 'projectsql1 (SQL)' database is expanded, showing a list of 'External tables' including 'dbo.dim\_date', 'dbo.dim\_rider', 'dbo.dim\_station', 'dbo.fact\_payment', 'dbo.fact\_trip', 'dbo.staging\_payment', 'dbo.staging\_rider', 'dbo.staging\_station', and 'dbo.staging\_trip'. The 'dbo.fact\_payment' table is highlighted with a red box. The main area shows a 'SQL script 1' editor with a query that counts rows for various tables. The 'Results' tab is active, displaying a table with two columns: 'count' and 'Table\_Name'. The results are as follows:

| count   | Table_Name   |
|---------|--------------|
| 36760   | FACT_Rider   |
| 839     | FACT_Station |
| 4584922 | FACT_Trip    |
| 1946607 | Fact_Date    |
| 1946607 | FACT_Payment |



The SQL script for the above output:

```
SELECT TOP 1* FROM dbo.fact_payment
GO
SELECT TOP 1* FROM dbo.fact_trip
GO
SELECT TOP 1* FROM dbo.dim_rider
GO
SELECT TOP 1* FROM dbo.dim_station
GO
SELECT TOP 1* FROM dbo.dim_date
GO
```

From the Data section of synapse analytics we can see that our star schema data is uploaded into the data lake storage gen2:

Microsoft Azure | Synapse Analytics | projectsynapse-1

Search

ritikkumar7280@gmail.com  
DEFAULT DIRECTORY

Synapse live | Validate all | Publish all

Data

Workspace | Linked

Filter resources by name

- Azure Blob Storage 1
- Azure Data Lake Storage Gen2 2
- projectsynapse-1 (Primary - project...
- projectcontainer1 (Primary)
- star1container
- (Attached Containers)
- Integration datasets 2

projectcontainer1

New SQL script | New data flow | New integration dataset | Upload | Download | New folder | Select all | More

| Name               | Last Modified        | Content Type | Size     |
|--------------------|----------------------|--------------|----------|
| Schema_date        | 30/08/2025, 01:10:20 | Folder       |          |
| schema_payment     | 30/08/2025, 00:37:54 | Folder       |          |
| schema_rider       | 30/08/2025, 00:54:55 | Folder       |          |
| Schema_station     | 30/08/2025, 01:01:19 | Folder       |          |
| Schema_Trip        | 30/08/2025, 01:06:47 | Folder       |          |
| public.payment.txt | 27/08/2025, 19:32:58 |              | 88.1 MB  |
| public.rider.txt   | 27/08/2025, 19:32:52 |              | 8.4 MB   |
| public.station.txt | 27/08/2025, 19:32:51 |              | 61.4 KB  |
| public.trip.txt    | 27/08/2025, 19:33:27 |              | 524.7 MB |

Showing 1 to 9 of 9 cached items

Here we can see that our star schema data is uploaded into the storage account container:

Microsoft Azure Upgrade Search resources, services, and docs (G+/) Copilot ritikkumar7280@gmail.c... DEFAULT DIRECTORY (RITIKKUM...)

Home > projectstorageritik1 | Containers > projectcontainer1 Container

Search Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Shared access tokens

Manage ACL

Access policy

Properties

Metadata

projectcontainer1

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive) Only show active objects

Showing all 9 items

|                          | Name               | Last modified        | Access tier    | Blob type  | Size       | Lease state |
|--------------------------|--------------------|----------------------|----------------|------------|------------|-------------|
| <input type="checkbox"/> | Schema_Trip        | 30/08/2025, 01:06:47 |                |            |            | ...         |
| <input type="checkbox"/> | Schema_date        | 30/08/2025, 01:10:20 |                |            |            | ...         |
| <input type="checkbox"/> | Schema_station     | 30/08/2025, 01:01:19 |                |            |            | ...         |
| <input type="checkbox"/> | schema_payment     | 30/08/2025, 00:37:54 |                |            |            | ...         |
| <input type="checkbox"/> | schema_rider       | 30/08/2025, 00:54:55 |                |            |            | ...         |
| <input type="checkbox"/> | public.payment.txt | 27/08/2025, 19:32:58 | Hot (Inferred) | Block blob | 88.08 MiB  | Available   |
| <input type="checkbox"/> | public.rider.txt   | 27/08/2025, 19:32:52 | Hot (Inferred) | Block blob | 8.44 MiB   | Available   |
| <input type="checkbox"/> | public.station.txt | 27/08/2025, 19:32:51 | Hot (Inferred) | Block blob | 61.38 KiB  | Available   |
| <input type="checkbox"/> | public.trip.txt    | 27/08/2025, 19:33:27 | Hot (Inferred) | Block blob | 524.68 MiB | Available   |

....Complete....